

KitchenBuddy

A kitchen assistant that runs on a laptop

Project report

ISEN Méditerranée, engineering cycle

September 2026

A team of five: AI Researcher, Data Engineer, ML Engineer, DevOps Engineer, Scrum Master

App name: Happy Bite · Code: github.com/Infernoflamex/KitchenBuddy

Abstract

KitchenBuddy (app name: Happy Bite) is a kitchen assistant we built in six days. It keeps track of what is in the kitchen and when it goes off, reads supermarket receipts, suggests dinner from what is already there, writes recipes, and talks you through cooking one step at a time, hands-free.

One decision shaped every other: the generative model runs on our own hardware, not behind someone else's API. Our demo machine is a MacBook Air with 16 GB of memory, shared by the language model, the speech models and everything else. On that machine a model writes about 15 tokens per second, has a small working context, and makes mistakes that a cloud model would not.

We compared the four ways of adapting a model to a task: prompting, parameter-efficient fine-tuning, full fine-tuning and retrieval-augmented generation (RAG). We chose **structured prompting, RAG, and strict checking of everything the model returns**. The knowledge the assistant needs is a household's stock, which changes every day, and a corpus of 2.1 million recipes. Neither belongs in model weights. The final model is **Gemma 4 E2B** in 4-bit, served locally through LM Studio. Retrieval uses a SQLite full-text index. A LoRA fine-tuning pipeline is written and self-tested, but we kept the training run as a second stage and explain why with numbers.

This report explains what we built, why we chose each component, how the pieces fit together, what we measured, what still does not work, and what we learned.

1. The problem and the constraint

1.1 The task

It is seven in the evening. You open the fridge: half a courgette, some goat's cheese, rice, and two chicken thighs that have to be used tomorrow. You are tired, you have twenty minutes, and you do not want to shop. A recipe site cannot help, because it does not know what is on your shelves. A recipe you know by heart does not help either, because it needs the one thing you have run out of. Half an hour later something gets thrown away.

Stated plainly, the task is this: given a household's own inventory, what is about to spoil, and the time the cook has, produce a recipe that uses those ingredients, in a machine-readable form the application can act on, and then guide the cook through it by voice.

It is a demanding task because it exercises most of a generative system at once. The knowledge needed is large, changes daily and must be traceable, which is the retrieval case. The output must be structured, which is the decoding and validation case. The input is speech, which adds two more models. And the whole thing has to run inside a fixed memory budget, which is where the engineering is.

1.2 The constraint that decided everything

The model runs on our own machines, on models published openly rather than through a paid API. That sounds like a detail. It turned out to be the whole project.

A cloud model hides most of the engineering: huge context, fast answers, reliable JSON, tool calling that just works. A small model on a laptop behaves very differently. It reads a prompt of a few thousand tokens before it writes a single word. It writes about 15 tokens per second. It sometimes invents an ingredient, or asks for a quantity nobody could eat. So a large part of our work was not choosing a model. It was making a small model dependable enough to use while standing at the stove.

1.3 What we set out to deliver

- Track the household's stock: quantity, purchase date, storage place, and an estimate of when each item goes off.
- Fill that stock from a photographed receipt, or by voice ("I bought 300 grams of chicken").
- Suggest and write recipes from what is actually in the kitchen and the time available.
- Guide the cook step by step, mainly by voice, in English or French.
- Justify each technical choice with numbers against at least one real alternative.

1.4 What we actually shipped

Everything above works in the version we demo, on one MacBook Air. Cooking, including the voice, needs no internet at all; only the app-wide hands-free button outside cooking mode does, for the reason explained in 4.8. Some parts are rougher than we would like, and section 10 lists them honestly. The fine-tuning run is the one objective we deliberately postponed.

2. How we worked

2.1 Roles

We were five, one per role: AI Researcher, Data Engineering, ML Engineer, Integration and Scrum Master. We call the Integration role DevOps Engineer in this report because that is what the work turned out to be: environments, model servers and deployment as much as wiring. With six days and five people the coordination role earned its place. Most of what went wrong in the first two days was organisational rather than technical (section 2.3), and it was the Scrum Master who ran those down.

Role	What it meant in practice
AI Researcher	Compared adaptation strategies and models, did the LoRA arithmetic, chose sampling settings, tested candidate models on the Mac.
Data Engineer	Chose and compared datasets, exported the 2.6 GB recipe corpus, grew the ingredient catalogue to 3,052 entries, built the Open Food Facts product index and an embedding prototype in ChromaDB.
ML Engineer	Built retrieval, the recipe checks and the "template first" path, wrote the fine-tuning scripts and the diagnostic tools.
Integration (DevOps Engineer)	FastAPI backend, React frontend, local model server setup, voice integration, deployment, and the macOS and Windows guides.
Scrum Master	Ran the six-day plan and the daily check-ins, tracked what was blocked and cleared it: repository access, the duplicated folder tree, the Python version pinning and who was testing on which machine. Kept the report and the demo on the calendar.

Table 1. Team roles.

2.2 Six days

Day	What we did	In the Git history
11 Sept	Framed the need, picked datasets, designed the stock and recipe formats, first FastAPI and React skeleton, first prompts.	Initial commit, "Phase 1"
12–14 Sept	Local voice: speech-to-text, text-to-speech, wake word. One API for LM Studio, Ollama or a cloud model.	"Phase 2 - Voice"
14–15 Sept	Cooking mode, the recipe index, template-first recipes, receipt reading, recipe checks.	"Phase 3 - Redo", "Phase 3 - Part 2"
15 Sept	Catalogue builder, corpus importer, Open Food Facts importer, fine-tuning scripts.	"Data related scripts"
16–17 Sept	Fixes, guides for macOS and Windows, testing on the demo Mac, receipt camera, report and demo.	"Phase 1-2-3 Correction...", "Phase 4"

Table 2. Our plan, matched to the commit history.

2.3 What went wrong along the way

Most of our lost hours had nothing to do with AI.

- **One repository on one person's account.** The code started on a member's GitHub account from an earlier competition. We had to clone it, push it to a second repository, and learn the hard way to check which remote we were pushing to.
- **A folder that existed twice.** An archive was extracted twice, which produced duplicated folders and a nested shared/shared directory. Imports in the frontend broke in ways that looked like code bugs. The fix was to flatten the tree. The lesson was to check that a file really exists before blaming the code that imports it.

- **Mac and Windows in the same team.** The Mac runs MLX models on its GPU; Windows runs GGUF models through llama.cpp. The voice stack differs too. This forced us to make every model a configuration setting instead of a dependency, and to write two installation guides.
- **"The model is broken" when it was not.** Several times the model looked broken and the real cause was configuration. It had been loaded with a 4,096-token window while we sent it 34,000 tokens. A reasoning model had its thinking mode on and spent 26 minutes before writing one recipe. Sampling settings tuned for one model stayed behind when we switched to another.
- **Python versions.** Some voice and image libraries publish nothing for Python 3.13 or 3.14, so we pinned 3.11 and 3.12.

We came out of this with four habits: measure before diagnosing; write next to each setting which model it was tuned for; treat model output as untrusted input; and when a deterministic check can do the job, prefer it to hoping the model gets better.

3. The big decision: prompting, RAG or fine-tuning?

3.1 What we had to work with

- **Data that changes every day and is different for every household.** The stock in a fridge cannot live in a model's weights.
- **A lot of existing knowledge.** Millions of recipes already exist, written by people who cooked them. We need to find the right one and follow it, not memorise a blurry average of all of them.
- **One laptop.** 16 GB of shared memory for inference, and no dedicated GPU for training.
- **A person waiting at the stove.** A spoken answer has to start within a couple of seconds.

3.2 Comparing the four options

Strategy	Fresh data?	Cost to update	Training data needed	Doable on our laptop?	What it would fix for us
Prompting only	Yes, in context	None	None	Yes	Tone, rules, output format
RAG	Yes, re-index	Minutes	None, just a corpus	Yes	Knowledge, sources, grounding
LoRA / QLoRA	No, frozen	1–2 h per run on one GPU	Hundreds to thousands of examples	On NVIDIA yes; on our Mac only a 1.5B model	Format, ingredient ids, style
Full fine-tuning	No, frozen	Days, several GPUs	Tens of thousands	No	Same as LoRA, far more expensive

Table 3. The four strategies against our constraints.

The deciding argument was freshness. The rule of thumb is simple: retrieval for knowledge that changes or must be cited, fine-tuning for behaviour. A fridge changes every day. A recipe should come with its source. Both point to retrieval. Fine-tuning a model on the contents of a fridge would make no sense; putting the fridge in the prompt costs nothing.

3.3 Doing the maths on LoRA

We still wanted to know what fine-tuning would cost, so we counted the parameters. LoRA freezes the original weights and trains two small matrices per adapted layer. For a matrix of size $d_{in} \times d_{out}$ at rank r , it trains $r \times (d_{in} + d_{out})$ parameters.

For one 4096×4096 attention matrix, full fine-tuning updates 16.8 million parameters; LoRA at rank 8 trains 65,536, about 0.4 %. We applied the same formula to the configuration in our training script: rank 16 on all seven projection matrices of every layer.

Base model	Layers	Hidden / MLP / key-value width	Trainable parameters at $r = 16$	Share of the model
Qwen2.5-7B-Instruct	28	3,584 / 18,944 / 512	40.4 million	0.53 % of 7.61 billion
Qwen2.5-1.5B-Instruct	28	1,536 / 8,960 / 256	18.5 million	1.2 % of 1.54 billion
Gemma 4 E2B, the model we serve	35	1,536 / 6,144 / 256	18.9 million	0.37 % of 5.1 billion

Table 4. LoRA size for our training configuration, computed from each model's published architecture.

Forty million parameters train in an hour or two on one consumer NVIDIA GPU. A full fine-tune of the same 7B model would need gradients and optimiser state for all 7.6 billion weights. With AdamW in mixed precision that is roughly 16 bytes per parameter, so about 120 GB of GPU memory. That was never an option for us.

The last row deserves a word, because the two numbers can look inconsistent. Our training scripts target Qwen2.5, which was our model when they were written; the model we ended up serving is Gemma 4 E2B. The arithmetic is the same method applied to a different shape: on Gemma, rank 16 across the same seven projections in 35 layers is **18.9 million trainable parameters, 0.37 % of its 5.1 billion**, and a full fine-tune of it would need about **82 GB**. Pointing the scripts at Gemma is a base-model change and a chat-template check, and it is the first task of section 7's second stage. The figure to quote for this project is Gemma's.

3.4 What we decided

We built on three things.

1. **Structured prompting.** One system prompt per task: chat, cooking mode, recipe writing, recipe import, receipt reading. Each states the role, the rules of the kitchen and the expected output format.
2. **RAG.** Before writing a recipe, the server searches the corpus for similar recipes, or for the real recipe of a dish the user named. The user's stock is sent with every request.
3. **Strict checking.** Everything the model returns is treated like untrusted user input and checked before the app uses it (section 5.4).

LoRA stays as a second stage, and not only for lack of a GPU. Gemma 4 E2B already cooks competently in general: techniques, timings, what goes with what. What it lacks is this household's kitchen tonight and the exact recipe to follow, and both change daily, so they belong in retrieval rather than in weights. The scripts exist and would mainly teach the model our recipe format and ingredient ids. Section 7 explains why we did not run them this week.

3.5 Where each argument is made

Every choice above is defended somewhere with a number rather than a preference. This is where each argument is made.

The question	Where it is answered
How the model is told what to do	The system prompts and the kitchen rules, and the ingredient context injected per request: sections 5.3 and 5.4
How the next word is chosen	Temperature, top-p and schema-constrained output, with our values and reasons: section 4.6
What training on our own data would involve	The LoRA pipeline, its configuration and its parameter count: sections 3.3 and 7
What that would cost, exactly	Counted for our own configuration: 18.9 million trainable parameters on the model we serve, against about 82 GB of GPU memory for a full fine-tune: section 3.3
Why retrieval rather than training	The retrieval design and the freshness argument that decided the strategy: sections 3.2 and 6
How we know whether it works	Why the usual training measure is useless here, and the task metrics we used instead: section 9
What could go wrong for a user	The risk register and the compliance review: sections 10.2 and 10.3

Table 5. Each argument, and where it is made.

4. The machine, the model, and the settings

This is the heart of the project, so it is written in full: what the hardware can do, what the model actually is, what 4-bit means, which runtime runs it and why, how much memory each piece takes, and why each setting is where it is. Section 13 explains the terms in plain words.

4.1 The machine: Apple Silicon and unified memory

Our demo machine is a **MacBook Air with an Apple M4 and 16 GB of memory**, 10 cores (4 performance, 6 efficiency). Apple Silicon is Apple's own ARM processor family: the M series in Macs, and the A series in iPhones, which Apple brands **Bionic**. They share an architecture, so what is true of an iPhone A-series chip is broadly true of an M4, at a different scale. Two properties matter here.

- **Unified memory.** There is no separate graphics memory. The CPU, the GPU and the Neural Engine all address the same 16 GB. A model's weights do not have to be copied to a graphics card, which is why a laptop with no dedicated GPU can run a language model at all. It also means every gigabyte the model takes is a gigabyte the browser, the speech models and macOS cannot have.
- **High memory bandwidth.** Generating a token requires reading every active weight from memory. Token generation on a laptop is therefore limited by memory bandwidth far more than by arithmetic, which is the real reason a smaller or more compressed model is faster.

The Neural Engine is used by the operating system's own features, not by our stack: MLX runs on the GPU.

Everything the AI layer needs comes out of the same 16 GB:

What	Memory	Where the number comes from
Gemma 4 E2B, 4-bit weight file	4.34 GB on disk	The build we run. Less than this is resident while it runs: see 4.3
KV cache at our 8,192-token window	73 MB	Computed from the model's configuration (4.5)
Parakeet 0.6B, speech recognition	about 1.2 GB	Weight size
Kokoro 82M, speech synthesis	about 0.3 GB	Weight size
Total for the AI layer	about 5 GB	Leaving roughly 11 GB for macOS, Chrome and everything else

Table 6. The memory budget on the demo Mac.

4.2 What Gemma 4 E2B actually is

Numbers below come from the model's own configuration file on Hugging Face, not from a blog post.

Property	Value	What it means for us
Parameters	2.3 billion effective, 5.1 billion in total	Per-Layer Embeddings hold a large share of the weights in lookup tables that are cheap to read, so the model costs about what a 2.3 B model costs to run while knowing what a 5 B model knows
Layers	35	Depth of the stack; every layer's weights are read for every token
Attention heads	8 query heads, 1 key/value head	Multi-query attention: all 8 heads share one set of keys and values, so the cache that grows with the conversation is 8 times smaller than it would otherwise be
Head dimension	256	Sets the size of that cache: $2 \times 1 \times 256 \times 2$ bytes = 1 KB per token per layer
Attention pattern	28 sliding-window layers (512 tokens), 7 full-attention layers	Only every fifth layer looks at the whole conversation; the rest look at the last 512 tokens. This is what keeps the cache small (4.2.1)
Vocabulary	262,144 tokens	A large vocabulary means fewer tokens per sentence, which means fewer passes through the model
Context window	131,072 tokens	What the model supports. Not what we use: see 4.5
Published precision	16-bit	Two bytes per weight as published, which is what quantisation shrinks

Table 7. Gemma 4 E2B, from its published configuration.

4.3 4-bit quantisation: what it is and why we use it

A trained model stores each weight as a 16-bit number: two bytes. **Quantisation** stores it in fewer bits instead. Four-bit quantisation keeps only 16 possible levels per weight. On its own that would be far too coarse, so the weights are quantised in small groups (64 weights at a time in MLX): for each group the tool records the smallest and largest value as a scale and an offset in higher precision, and then each weight is the nearest of 16 steps between them. At inference the group is expanded back to a real number on the fly. The error is small because weights inside a group are of similar magnitude.

The point is the size. Weights dominate what a model costs to hold and to read:

Precision	Weight file	Fits in 16 GB beside the voice models and a browser?
16-bit, as published	10.25 GB	No
8-bit	about 5 GB	Tight: little room for speech models or the browser
4-bit, the build we run	4.34 GB	Yes

Table 8. Why the 4-bit build is the one that fits. Sizes read from the published repositories, not computed. The 4-bit file is bigger than a naive 2.5 GB because scales, offsets and some tensors stay at higher precision: about 6.8 bits per parameter on average.

A file size is not a memory requirement, and here the difference is large. Roughly 2.4 of the 5.1 billion parameters are per-layer embedding tables: 262,144 vocabulary entries for each of 35 layers. Those are lookups, read a row at a time for the tokens in front of the model, not weights that have to stay resident. That is what Google means by "2.3 billion effective parameters", and it is why sensible figures for this model's working memory are a few gigabytes rather than the size of its weight file. The budget in 4.1 uses file sizes because those are what we can verify; what is actually resident is lower, and grows with the context window through the cache in 4.5.

The cost is accuracy. Quantisation is lossy, and 4-bit loses more than 8-bit. Published work on the method we rely on (QLoRA, Dettmers et al., 2023) shows the loss is small for well-chosen group sizes, and our own check was task-level rather than academic: the 4-bit build's recipes passed our validators as often as the alternatives we tried, so we spent the memory on the speech models instead. On this machine the choice is not really open. The full-precision model does not fit at all, and an 8-bit build would leave the voice fighting the browser for what is left.

4.4 MLX, GGUF, LM Studio and Ollama

A model file is not a program. Something has to load those weights and run the arithmetic on this particular chip. There are two realistic options on a Mac.

- **MLX** is Apple's own array framework for Apple Silicon, in the same family of ideas as NumPy or PyTorch but built for unified memory: arrays live in memory both the CPU and GPU can read, with no copying between them, and the heavy operations are compiled to Metal, Apple's GPU language. Models are published in MLX format ready to run, and quantised with MLX's own scheme.
- **llama.cpp** is the portable C++ engine behind most local model tools, using the **GGUF** file format. It runs on Apple Silicon through Metal too, and on Windows and Linux with CUDA or Vulkan, which is what makes it the format our Windows team members use.

We serve the model with **LM Studio** on the Mac. Its reason is specific: on Apple Silicon, LM Studio runs MLX builds, and MLX is written for precisely this chip. Community benchmarks in 2026 put MLX roughly 15 to 30 % ahead of llama.cpp on throughput on Apple Silicon at similar memory use. We did not reproduce that number, so we treat it as a reason rather than a result.

Why not Ollama, which is otherwise the friendlier tool? Ollama added an MLX backend in version 0.19 (March 2026), but it falls back to llama.cpp below 32 GB of unified memory. Our Air has 16 GB, so on our machine Ollama would run the GGUF build through llama.cpp, which is the thing we were trying to avoid. Ollama also has a real advantage we gave up: it honours options sent with the request, so the context window, the keep-alive and the thinking switch can all be set from our configuration file, while LM Studio validates the request body and rejects those fields with a 400, which the backend detects,

drops and retries. On Windows, where MLX does not exist, the team uses Ollama for exactly that reason.

The application itself is indifferent. It speaks the OpenAI chat-completions dialect to whatever is listening, so LM Studio, Ollama and a cloud provider are one line of configuration apart. That decision is what let four people with different machines run the same code.

4.5 The context window and the KV cache

A transformer does not re-read the conversation for every new token. It keeps the keys and values it computed for every previous token in a **KV cache**, and a new token attends to that. The cache is what makes generation fast and what makes long conversations expensive, because it grows with every token.

For this model the arithmetic is unusually friendly, and it is worth doing because it explains our settings. With one key-value head of dimension 256 at 16 bits, one token costs $2 \times 256 \times 2$ bytes = 1 KB per layer. Multiply that out:

Situation	KV cache
8,192 tokens, as we run it	73 MB (7 full-attention layers over the whole window, 28 sliding layers capped at 512 tokens)
8,192 tokens, if every layer were full attention	294 MB, four times more
131,072 tokens, the model's full window	about 0.95 GB

Table 9. What the conversation costs in memory, computed from the model's configuration.

So the cache is not why we cap the prompt. **Time** is. Every token of prompt must be read by the GPU before the first token of the answer appears, and on this machine that is the largest part of a short wait. We therefore cap prompts at 8,192 tokens, reserve 35 % of that for the answer, and trim the ingredient-id list to fit (section 5.3). We also set the same 8,192 in LM Studio rather than leaving the model loaded at its full 131,072: a window the app never uses buys nothing and reserves memory we would rather give to the voice.

4.6 Decoding settings, and where they are set

Once the model has produced its scores for the next token, the **decoding** settings decide which one is chosen. The mechanics are standard; these are our values and our reasons.

Setting	Vendor default	Ours	Why
Temperature	1.0	0.2 for recipes, 0.3 for chat, 0 on a retry	Temperature flattens or sharpens the distribution. Conversation benefits from variety; a recipe in JSON with catalogue ids does not. Low temperature is the standard lever for constrained output
Top-p	0.95	0.95	Kept: with the temperature already low, clamping the tail twice makes answers stilted
Response format	free text	JSON schema	The server is given the recipe schema and only emits tokens that keep the JSON valid, which turns malformed output from unlikely into impossible

Setting	Vendor default	Ours	Why
Reply length	unlimited	3,000 tokens for a recipe, 220 for a spoken answer at the hob	A recipe is about 700 tokens of JSON; the rest is headroom. A spoken answer must be short or it is unusable
Context overflow	truncate middle	never reached	We size the prompt ourselves, so the runtime never has to decide what to throw away

Table 10. Decoding settings.

One practical note that cost us time: **the values in LM Studio's own settings screen are defaults for requests that do not specify them.** Our backend sends temperature, top-p and the response format with every request, so those app-level numbers are overridden. The settings that must be right in LM Studio are the ones the API cannot set on this server: the context length the model is loaded with, and the reasoning switch below.

4.7 Why reasoning is switched off

Recent models are often **hybrid reasoning** models: before answering, they generate a private chain of thought, usually inside <think> tags, and only then the answer. It genuinely helps on maths and multi-step logic. It is wrong for this app, for three reasons.

1. **It is paid for in tokens, at our token rate.** Thinking tokens are generated exactly like answer tokens. At about 15 tokens per second, a thousand tokens of deliberation is more than a minute before the recipe starts. We measured one Qwen 3.5 recipe that spent **26 minutes** thinking.
2. **It competes with the answer for the reply budget.** The thinking happens inside the same reply, so a 3,000-token budget can be spent on deliberation and leave the recipe truncated halfway through step six. That is exactly what we saw before the budget was raised.
3. **The task does not need it.** Our hard problems are not reasoning problems: they are format and grounding problems, and we solve those with a schema and with checks (section 5.4). A cook at the hob wants the answer, not the working.

Turning it off is not one switch. The app sends the standard field with every request (chat_template_kwargs with enable_thinking false), which Ollama and llama.cpp honour and LM Studio rejects with a 400; the backend notices, drops the field and asks again, so on LM Studio it must also be off in the model's own settings, in its prompt template. Whatever gets through, any <think> block that still arrives is stripped before the JSON is parsed. Our diagnostic script reports whether reasoning is on, because a model that is thinking looks exactly like a model that has hung.

4.8 Voice

Job	What we use	Alternative we considered	Why
Hearing (speech to text)	Parakeet TDT 0.6B on the Mac GPU; Whisper-base on CPU elsewhere	Whisper-small	Whisper-small makes about half the errors of Whisper-base but is 3× larger and about 2× slower. On a CPU, speed won. On the Mac, Parakeet is faster than both.
Speaking (text to speech)	Kokoro-82M	Piper	Piper is lighter and faster but sounds robotic. For something that talks to you

Job	What we use	Alternative we considered	Why
			for twenty minutes, voice quality won. A short phrase takes 0.15 to 0.25 s to generate on the Mac.
Wake word, outside cooking	The browser's own speech recogniser, listening for "Bob"	openWakeWord on device	Spotting a word in a continuous stream is a different job from transcribing a clip. Our speech model transcribes a recording that someone has already decided to make; something has to decide when to start recording. The browser's recogniser does that for free, with no model to download, and it was a day's work. The price is that in Chrome it is a cloud service, so it needs internet and sends that audio to Google. An on-device wake model (openWakeWord in ONNX) removes both problems and is not built yet.

Table 11. Voice components.

If Kokoro fails to load, the app falls back to Piper and then to the operating system's voice, so the assistant never goes silent. French has only one Kokoro voice, which is the main weakness here.

4.9 Data

Need	Chosen	Rejected	Why
Recipes	Kaiser1308/CookingRecipes: title, ingredients, directions, source link and ingredient names in separate columns; 2,119,522 recipes indexed	Khushis04/recipe-suggester-data; corbt/all-recipes	The largest structured set, and it keeps the source of every recipe. The first alternative has no source. The second stores each recipe as one block of text, which breaks retrieval when ingredients get separated from their steps.
Products on receipts	Open Food Facts, products sold in France: 483,210 products, 329,783 of them linked to one of our ingredients	Letting the model read receipts alone	A till line like "CRF LT DEMI ECR 1L" is an abbreviation puzzle. Searching real product names solves it better than a small model guessing.
Ingredient vocabulary	Our own catalogue: 3,052 ingredients with name, category and unit	Free-text ingredient names	Ids are what make checking possible. An invented id can be caught and dropped.
Substitutions and nutrition (prototype)	oraclemangle/historical-culinary-substitutions; omid5/usda-fdc-foods-cleaned	lishuyang/recipepairs; the full Open Food Facts nutrition export	Ingredient-level swap rules and a lighter nutrition table. Indexed in our ChromaDB prototype, not yet in the app.

Table 12. Datasets.

One honest caveat: the recipe corpus is mostly American home cooking in English. Its most typical carbonara uses bacon and butter. The assistant follows what the corpus says a dish is, and we would rather say so than pretend otherwise.

5. How the application works

5.1 What the user sees

The app has five screens: Today (tonight's suggestion), Kitchen (stock and expiry), Recipes (saved recipes), Receipt and Shopping. On top of them sit the chef chat and cooking mode. A typical evening looks like this:

1. You come home and photograph the receipt, or say "Bob, I bought 300 grams of chicken". The stock updates with an expiry estimate.
2. You say "Bob, I'm drained, give me 15 minutes". The app writes one dinner from what is already in the kitchen, with nothing to buy, and opens it.
3. You press Start cooking. The chef reads one step at a time, tells you what to look and listen for, starts timers, and answers questions about the step. You say "Bob, next" to move on, or tap the Talk button and just say "next": the name exists so that the radio and the conversation do not move the recipe on, and the button is for when your hands are free and saying it again is the annoying part. Both paths transcribe on the machine.

5.2 Architecture

We split the app into two layers. **CORE** is everything that does not need a model: stock, expiry, shopping list, recipe book, cooking history. It runs entirely in the browser and stores data locally, so it still works when the server is off. **EXTRA** is the AI layer: chat, recipe writing, cooking mode, receipt reading, voice. Each EXTRA feature switches itself off and says why if its model or package is missing.

The server keeps no household data. The browser sends the stock with each request, and nothing is stored server-side. The model can change the kitchen only through eight tool calls (add stock, adjust stock, set expiry, add an ingredient, save a recipe, add to shopping, set a timer, open a screen), and each call is checked against the catalogue. In short, the model can only do what you could already do by tapping.

```
voice or text
|
v
Parakeet / Whisper (speech to text)
|
+--> stock and profile (from the browser)
+--> ingredient ids (catalogue, trimmed to fit)
+--> similar or reference recipes (SQLite full-text search)
|
v
Gemma 4 E2B in LM Studio (OpenAI-compatible API, on the laptop)
|
+--> recipe JSON --> clean and check --> retry once if wrong
+--> chat text, streamed --> sentence by sentence --> Kokoro (voice)
+--> tool call --> checked --> applied in the browser
```

Figure 1. What happens between a question and an answer.

5.3 Fitting everything into the context window

This is where local models break most often, so we handle it explicitly.

The number that matters is not the 128K on the model card but the window the model was actually loaded with, and how long the user is willing to wait. On a laptop, every token of prompt is read before

the first word of the answer. The server asks LM Studio or Ollama what window the model was loaded with, caps prompts at 8,192 tokens, and keeps 35 % of that for the answer.

The most expensive part of a prompt is the ingredient list. Sending all 3,052 ids would be about 34,000 tokens. The server keeps what is in the kitchen first, then seasonings, then as much of the rest as fits. Anything that does not fit can still appear in a recipe as plain text. We learned the limits by breaking them. With a 4,000-character ingredient list in the recipe prompt, Gemma started returning recipes with no ingredients at all, so chat and recipe writing now have separate budgets.

Finally, recipe requests send the JSON schema to the model server, which then only allows tokens that keep the JSON valid. Malformed JSON went from occasional to impossible.

5.4 Never trusting the model's output

Everything the model returns goes through checks written in plain Python before the app uses it:

- **Cleaning.** Invented ingredient ids are dropped. Impossible quantities are brought down to normal portions, and the recipe says so. Seasonings are sorted according to the catalogue, not the model's opinion.
- **Problem detection.** Do the step times add up to the time the person has? Does a step mention a food that is not in the recipe? Is a sauce used before the step that makes it? Does the oven get turned on before something is baked?
- **One retry.** If problems are found, the model gets the exact list and one more try. If the second attempt is still wrong, the better one is shown with a "check this before you start" box.
- **Named dishes.** When the user asks for a specific dish, the result is compared with the real corpus recipe it was based on, and sent back if a key ingredient or cooking method disappeared.

These checks came from real failures: 380 g of salt for three people, 1.7 kg of courgettes, a litre and a half of olive oil, a step lasting 255 minutes.

5.5 Reading a receipt

A receipt goes through three stages. **Tesseract**, an open-source OCR engine running locally, turns the photo into lines of text. Each line is then **looked up** in the Open Food Facts index: abbreviations are expanded, the database is searched by word fragments so "ECR" can find "écrémé", and candidates are scored word by word. Finally **Gemma** reads each line together with what the lookup found and decides the ingredient and quantity. When the model and the database agree, confidence goes up. When they disagree, the line is marked "Check" for the user. Corrections the user makes are remembered for the next receipt.

Getting a usable photo was its own small project. The first version used a file input, which on a laptop opens a file picker, not the camera. We replaced it with a live camera view. That view looked zoomed in and blurry, and measuring the camera showed why: we had asked for a 4K widescreen frame the MacBook camera does not have, so Chrome gave us a narrow 1080 × 1920 slice of the image. Asking the camera for its own largest frame gives 1552 × 1552 with the full field of view. We also added a review step so a blurry photo can be retaken before a minute is spent reading it.

6. Retrieval in practice

6.1 Why full-text search and not embeddings

Textbook retrieval uses embeddings in a vector database. Our Data Engineer built exactly that as a prototype: bge-small-en-v1.5 embeddings stored in ChromaDB, with separate collections for recipes, substitutions and nutrition. It works. We still shipped something simpler.

Criterion	Embeddings in ChromaDB (prototype)	SQLite FTS5 with BM25 (shipped)
Indexing the 2.1 million recipes of Kaiser1308/CookingRecipes	One pass of bge-small-en-v1.5 per recipe: hours on a laptop, about 15 GB of disk. The prototype stops at 50,000 recipes per collection.	Built in a background thread while the app already runs; 2.9 GB index; rebuilt only when the corpus changes.
Extra dependencies	PyTorch, sentence-transformers, ChromaDB	None beyond Python itself
Understands meaning	Yes: "something cosy with chicken" finds stews	No, it matches words. Helped by stopwords, accent folding and dish detection.
Query time	A few milliseconds on 50,000 rows	3.3 s at first; tens of milliseconds after the fix in 6.2

Table 13. The two retrieval engines, measured on the demo Mac.

The size of the corpus against the size of the laptop decided it. Our queries are mostly lists of ingredients plus perhaps a dish name, which word matching handles well. Semantic re-ranking of the best word matches is our planned next step.

6.2 From 3.3 seconds to a few milliseconds

Our first version was slow for an instructive reason. The query was a sentence plus the whole stock list, so it always contained words like "salt", which appears in about half of all recipes. The ranking algorithm had to score a million recipes on every request: 3.3 seconds before the model even started.

The fix: drop words that cannot tell recipes apart (grammar, units, anything in more than a quarter of the corpus), keep the five rarest words, and require all of them. If too few recipes match, drop the most common of the five and try again. Search now takes tens of milliseconds, and the results are better: a recipe with chicken and rice and black beans, rather than whichever recipe says "chicken" most often.

6.3 Named dishes and "template first"

Early on, "make mochi" returned a bruschetta. The ingredient code "oliveoil" in the stock list happened to be rarer in the corpus than the word "mochi". So the server now recognises when a request names a dish. A word counts as a dish when the corpus uses it in recipe titles at least as often as in ingredient lists: "carbonara" 55 times more, "chicken" much less. For a named dish, the server picks the most typical version, the one whose ingredients most other versions share, and asks the model to follow it.

For an ordinary dinner, the server first looks for a corpus recipe the kitchen can already make: at least 72 % of its ingredients in stock. If it finds one, the model only converts that recipe into our format instead of inventing a dish. Converting is easier and more reliable for a small model. There is an exception, which we found by measuring: for a 15-minute dinner, converting a corpus recipe took 48 seconds and then failed the time check, while writing a short recipe from scratch took 20 to 25 seconds. Quick dinners skip this step.

7. Fine-tuning: built, not run

The training folder contains a complete LoRA pipeline using Hugging Face transformers, peft, trl and datasets:

1. **build_dataset.py** streams the recipe corpus without loading it into memory and writes chat-style training examples, split into training and validation sets.
2. **build_app_dataset.py** does the same in the app's own recipe format and reports how many source ingredients map to a catalogue id. That figure is also a data-quality measure.
3. **train_qlora.py** trains LoRA at rank 16 (alpha 32, dropout 0.05) on all attention and MLP projections: two epochs, effective batch size 16, learning rate 2e-4. On NVIDIA it loads the 7B model in 4-bit (QLoRA). On the Mac it trains the 1.5B model without quantisation, because the 4-bit library only supports NVIDIA.
4. **test_model.py** runs held-out prompts through the tuned model and through the untuned baseline, and reports the share of answers that are valid recipes.
5. **export_ollama.py** merges the adapter into the model so it can be served through the same API as any other model.

Why we did not run it. Our only capable GPU was also the demo machine. Schema-constrained decoding and the recipe checks had already removed most of the failures a fine-tune would target. Each training run would take hours we preferred to spend on the parts the user touches. The README says plainly that the pipeline has self-tests but has not been run end to end.

There is also a mismatch we should name. The scripts target Qwen2.5, our original choice, while the final model is Gemma 4 E2B. Pointing them at Gemma means changing the base model and checking its chat template. That would be the first task of a second stage, followed by measuring the valid-recipe rate against the untuned baseline.

8. Integration and deployment

Layer	Technology
Frontend	React 18 and Vite; data stored in the browser (IndexedDB); unit tests for the non-AI logic
Backend	Python 3.11/3.12, FastAPI, streamed answers (server-sent events)
Language model	LM Studio (MLX on Mac) or Ollama (GGUF), both exposing an OpenAI-compatible API; a cloud provider is possible but not used in the demo
Voice	Parakeet or Whisper for hearing, Kokoro for speaking; weights downloaded from Hugging Face on first use
Retrieval	SQLite FTS5
Receipts	Tesseract OCR (French and English), Open Food Facts SQLite index, the language model
Running it	Three processes during development (model server, backend, frontend); one address once the frontend is built

Table 14. Technology stack.

A few integration decisions saved us repeatedly:

- **One API for every model.** The backend only speaks the OpenAI chat format to a local server. Switching between Llama 3.2 1B, Qwen2.5 3B, Qwen 3.5 and Gemma 4 E2B during the week meant editing one line of configuration.
- **Every feature optional.** A missing package turns one feature off and the status endpoint explains why. Team members with different machines could all run the same code.
- **Tools that measure.** `check_model.py` measures the model server. `audit.py` reads a recipe book like a cook would and lists mistakes. `check_voice.py` tests the voice chain end to end.
- **Privacy by default.** Everything runs locally. The server stores no household data, profiles contain no health data, and the recipe importer refuses private network addresses.

9. Testing and evaluation

9.1 Why not perplexity

Perplexity measures how well a model predicts a reference text. There is no reference text for "dinner from this fridge". A fluent recipe that uses hummus you don't have is a failure that perplexity cannot see. The same objection applies to the usual text-overlap scores, so we measured things tied to actual use.

9.2 What we measured

What	How	Result
Speed of the model	<code>check_model.py</code> against LM Studio	About 15 tokens/s for Gemma 4 E2B on the Air; a full recipe of about 700 tokens takes roughly 45–50 s (estimated from that rate)
Quick dinner	Timed on the demo Mac with a test fridge	20–25 s for a 15-minute recipe
Retrieval time	Timed per request	3.3 s before the query fix, tens of milliseconds after
Valid JSON	Server parse step	Malformed JSON no longer occurs with schema-constrained decoding
Hallucinated ingredients and portions	Recipe checks on every request	Caught and sent back once; remaining problems shown to the user, not hidden
Voice	Server logs	Voice models warm in about 9 s at start-up; a short phrase takes 0.15–0.25 s to synthesise
Receipt camera	Camera settings read in Chrome	1080 × 1920 cropped frame before, 1552 × 1552 full view after
Receipt data (Open Food Facts)	Database counts	483,210 products; 329,783 linked to a catalogue ingredient

Table 15. What we measured.

9.3 An experiment on the cooking assistant

While testing a rice recipe, we noticed the chef giving odd answers. Asked "I want to eat rice" on step 2 ("add the rinsed rice and tomato to the pot"), it answered from step 3: "keep simmering until the liquid is absorbed". We replayed the same three questions through the real prompt, three times each, changing one thing at a time.

Setup	Answer stayed on the current step
As the app sends it, with conversation history	0 of 3 runs
Same, with the recipe corrected	0 of 3 runs
No conversation history	3 of 3 runs
History, with the current step repeated inside the question	3 of 3 runs
No history, with the current step repeated inside the question	3 of 3 runs

Table 16. Does the chef answer about the right step? Gemma 4 E2B, temperature 0.3.

The cause is not the recipe. In our prompt, the current step appears once, at the end of the system prompt, and the conversation history follows it. A 2-billion-parameter model follows the conversation rather than a line far above it. Repeating the step inside each question fixed it in every run. We have not shipped that fix yet; it is the first item in section 10.3. We like this experiment because it is what evaluating on real usage looks like: a small, controlled test that turned a vague "the chef is confused" into a cause and a fix.

9.4 Incidents that changed the design

- **The 26-minute recipe.** A reasoning model thought for 26 minutes before answering. We now turn thinking off in every request, strip any reasoning text before parsing, and `check_model.py` reports whether reasoning is on.
- **The 437-second wait.** The HTTP client retried twice silently, so a four-minute failure looked like a twelve-minute hang. We now make one attempt with a long timeout and explain what to change when it expires.
- **34,000 tokens into an 8,192-token window.** We sent the whole ingredient catalogue. The server now asks what window the model has and budgets the prompt.
- **Prices read as quantities.** Gemma once added 2.31 courgettes and 7.45 cl of olive oil to the stock: those were the prices. A size printed on the line (1L, X6) now wins, and a quantity equal to the price is thrown out.

10. Limits and next steps

10.1 What does not work well yet

- **The chef can drift to the wrong step** after a few questions (section 9.3). The fix is known and tested but not shipped.
- **Recipe checks miss some errors.** A rice recipe with no water, whose step 3 simmers "until the liquid is absorbed", passed our checks. The checks catch wrong facts, not every cooking mistake.
- **Ingredient lists can be padded.** When the model lists the whole fridge as extras, each item becomes an ingredient with no quantity.
- **Retrieval matches words, not meaning,** and the corpus is mostly American home cooking.
- **French has one Kokoro voice.**
- **The app-wide hands-free button needs internet,** because Chrome's recogniser is a cloud service. Cooking mode does not: there the microphone is already open, the clip is transcribed

by our own model on the machine, and the name is matched in that local transcript. Since this version there is also a Talk button in cooking mode, which starts a turn without the name at all (section 5.1).

- **A laptop camera is about 2.4 megapixels with fixed focus.** A phone photo imported into the app reads better. iPhone HEIC photos are not supported yet.
- **Fine-tuning is not run,** and the scripts still target Qwen2.5.
- **One household, one browser.** Data is not synchronised between devices.

10.2 Risks we thought about

Risk	Where it shows up	What we do about it
Hallucination	Invented ingredients, quantities or steps	Retrieval from real recipes, schema-constrained output, checks with one retry, problems shown to the user
Unsafe advice	380 g of salt	Per-person caps; above the cap the amount becomes "to taste"
Over-reliance	Trusting a wrong time or heat	The prompt favours visual cues over clock times; time and heat checks
Bias	The corpus decides what a dish "is"	Stated openly; the reference recipe is visible
Privacy	What is in someone's kitchen	Local by default, no server-side storage, no health data

Table 17. Risk register.

10.3 Compliance

Anything that ships has to stand up to a licence and a privacy review, so we did one. We looked at three things: the licences of what we ship, the personal data we handle, and the advice the system gives.

Question	Where we stand
Model and software licences	Gemma 4 E2B is distributed under Google's Gemma terms, which permit local use and redistribution with conditions we have not breached: we ship no weights, the user installs the model themselves. Kokoro is Apache 2.0, Whisper MIT, Parakeet's MLX build follows its upstream terms, and the frameworks (MLX, llama.cpp, FastAPI, React) are permissive. Nothing in the repository bundles a model file
Dataset licences	The recipe corpus is used for retrieval and would be used for fine-tuning; it is a public Hugging Face dataset and each recipe keeps its source link, which is why we chose a corpus with provenance. Open Food Facts is published under the Open Database Licence, which requires attribution and share-alike on derived databases: our product index is derived and would carry that licence if we distributed it. We do not distribute it
Personal data (GDPR)	The household's data, its stock, its people and what they will not eat, stays in the browser on the device. The server is stateless and stores none of it. A profile is a name and a list of foods someone avoids; we deliberately collect no health data, no allergy diagnoses and no accounts. On the fully local setup nothing leaves the machine; the one exception is the hands-free wake word outside cooking mode, which uses the browser's cloud recogniser and is disclosed in the interface
Advice and safety	A cooking assistant gives advice that can make someone ill. Quantities are capped per person, so a model asking for 380 g of salt cannot reach the cook. Dry grains must be cooked. Raw flour must meet heat. Where a check fails twice, the recipe is shown with its problems rather than hidden. The app states that recipes are written by a model and should be read before they are trusted

Question	Where we stand
Allergens	We hold what a person will not eat and filter on it, but we make no allergen guarantee, and the interface does not pretend to: an ingredient list written by a model is not a label

Table 18. Compliance review.

Two things we would have to add before this could be used by anyone outside the team: an explicit statement of the model's limits at first run, and a way to export or delete the household's data in one action, which is a GDPR right we currently satisfy only by clearing the browser's storage.

10.4 Next steps

1. Repeat the current step inside every cooking-mode question, and re-run the experiment in 9.3 to confirm.
2. Add a check for recipes that simmer or absorb liquid that was never added, and stop turning unused extras into ingredients.
3. Point the fine-tuning scripts at Gemma 4 E2B, run them on an NVIDIA machine, and compare the valid-recipe rate with the untuned model.
4. Re-rank the best full-text matches with embeddings, and bring substitutions and nutrition from the prototype into the app.
5. Replace the browser wake word with an on-device model, and add HEIC support for phone photos.

11. What we learned

Measure before you argue. Our worst hours were spent guessing why the model was slow or wrong. A small script that prints tokens per second and whether the model is thinking answered in a minute questions we had been guessing about for much longer.

With a small model, most of the work is around the model. Choosing the model was a small part of the week. Making it dependable was most of it: budgeting the prompt, constraining the output, and checking every answer. Prompting, decoding and retrieval turned out to matter more day to day than anything about training.

Deterministic beats clever where it can. A check that says "these steps add up to 45 minutes and you have 20" is exact. A fine-tune that makes such mistakes less likely is statistical. We used the model for what only a model can do, and plain code for the rest.

The unglamorous parts decide whether a demo works. Repositories, folder structures, Python versions, a camera returning a cropped frame: none of it is AI, and all of it could have sunk the project. Next time we would name one person responsible for the environment from day one.

12. Conclusion

KitchenBuddy shows what running a generative model on student hardware really involves. The constraint pushed us toward prompting, retrieval and strict checking, rather than a fine-tune that looks attractive on paper but fits neither our compute nor knowledge that changes every day. We can defend that choice with numbers: the knowledge lives in a fridge and a corpus of 2.1 million recipes, not in

weights, and a fine-tune would cost 40 million trainable parameters and hours of GPU for problems our checks already catch.

The final system runs a 4-bit Gemma 4 E2B through MLX on a MacBook Air, alongside local speech recognition and synthesis, inside about 5 GB of a 16 GB machine. Each of the settings behind that sentence is defended somewhere in section 4 with a number rather than a preference: why 4-bit and not 8, why MLX and not llama.cpp on this machine, why an 8,192-token window when the model offers 128K, why reasoning is off, why decoding is cold. The system still makes mistakes, and we can name them because we measured them rather than assumed them.

13. The words we used

Short definitions of the technical terms in this report, in the order a reader meets them.

Term	What it means
Token	The unit a language model reads and writes: a word, a piece of a word, or a punctuation mark. Speed is measured in tokens per second, and context is measured in tokens
Parameters (weights)	The numbers learned during training. Reading all the active ones is what generating a token costs
Quantisation	Storing weights in fewer bits, in small groups with a shared scale. 4-bit keeps 16 levels per weight instead of the 65,536 a 16-bit number allows, roughly quartering the size at some loss of accuracy
Unified memory	Apple Silicon's single pool of memory shared by CPU, GPU and Neural Engine, with no copying between them
Apple Silicon	Apple's own ARM chips: the M series in Macs, and the A series in iPhones, which Apple brands Bionic
MLX	Apple's array and machine-learning framework for Apple Silicon, compiling to Metal, the Mac's GPU language. Models are published in an MLX format
GGUF	The model file format used by llama.cpp, the portable engine behind most local model tools, including Ollama
Context window	How many tokens of prompt and answer the model can hold at once. What matters in practice is the window the model was loaded with, and how long the user will wait for it to be read
KV cache	The keys and values kept for every token already in the conversation, so they are not recomputed. It grows with the conversation
Multi-query attention	All attention heads sharing one set of keys and values, which shrinks the KV cache. Gemma 4 E2B uses one key-value head for eight query heads
Sliding-window attention	Layers that attend only to the last N tokens (512 here) instead of the whole conversation, which keeps the cache small. Gemma 4 E2B makes every fifth layer a full-attention layer
Per-Layer Embeddings	Gemma's way of holding a large share of parameters in lookup tables read per layer, so a 5.1-billion-parameter model costs about what a 2.3-billion one costs to run
Temperature, top-p	Decoding settings. Temperature flattens or sharpens the distribution over the next token; top-p keeps only the most likely tokens that add up to p
Constrained decoding	Giving the runtime a JSON schema so it can only emit tokens that keep the output valid
Reasoning mode	A model writing a private chain of thought before its answer. Useful for logic, expensive in tokens, switched off here

Term	What it means
RAG	Retrieval-Augmented Generation: finding relevant documents at question time and putting them in the prompt, instead of training the knowledge into the weights
BM25	The ranking function full-text search uses to score how well a document matches a query
LoRA, QLoRA	Fine-tuning that trains two small matrices per layer instead of all the weights; QLoRA does it on a 4-bit-quantised base model
Perplexity	How surprised a model is by a reference text. A training-time measure, and not useful for judging a recipe (section 9.1)

14. References

Models

- Gemma 4 E2B-it (Google): <https://huggingface.co/google/gemma-4-E2B-it> ; 4-bit MLX build: <https://huggingface.co/mlx-community/gemma-4-e2b-it-4bit>
- Qwen2.5-7B-Instruct: <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct> ; Qwen2.5-1.5B-Instruct: <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- Qwen 3.5 9B: <https://huggingface.co/unsloth/Qwen3.5-9B-GGUF>
- Parakeet TDT 0.6B v3 (MLX): <https://huggingface.co/mlx-community/parakeet-tdt-0.6b-v3>
- Whisper-base and Whisper-small (OpenAI): <https://huggingface.co/openai/whisper-base>
- Kokoro-82M: <https://huggingface.co/hexgrad/Kokoro-82M> ; Piper: <https://github.com/rhasspy/piper>
- bge-small-en-v1.5: <https://huggingface.co/BAAI/bge-small-en-v1.5>

Datasets

- Kaiser1308/CookingRecipes: <https://huggingface.co/datasets/Kaiser1308/CookingRecipes>
- Open Food Facts: <https://world.openfoodfacts.org>
- oraclemangle/historical-culinary-substitutions: <https://huggingface.co/datasets/oraclemangle/historical-culinary-substitutions>
- omid5/usda-fdc-foods-cleaned: <https://huggingface.co/datasets/omid5/usda-fdc-foods-cleaned>
- Alternatives considered: Khushis04/recipe-suggester-data, corbt/all-recipes, lishuyang/recipepairs

Papers

- Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models.
- Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs.
- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
- Radford et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision (Whisper).
- Robertson and Zaragoza (2009). The Probabilistic Relevance Framework: BM25 and Beyond.

Tools

- LM Studio, Ollama, llama.cpp, MLX; Hugging Face transformers, peft, trl, datasets; FastAPI; React and Vite; SQLite FTS5; ChromaDB; Tesseract OCR.